Afterbit Ticket — Descrizione dello Stack Tecnologico

Documento tecnico di riferimento — Versione 2.3 (Ottobre 2026)

---

1. Panoramica del Progetto

Afterbit Ticket è una piattaforma web di gestione delle richieste di assistenza tecnica e amministrativa (Helpdesk/Ticketing) sviluppata con architettura Python/Django.

Consente ai clienti di:

• Aprire ticket categorizzati per area tecnica (hosting, posta elettronica, PEC, DNS, database, server, sicurezza web) con riconoscimento automatico del profilo utente connesso.

• Comunicare in modo bidirezionale con lo staff tramite un thread continuo di risposte e **invio di nuovi allegati** per ciascun messaggio senza dover aprire ulteriori ticket.

• Chiudere autonomamente i propri ticket in qualsiasi momento tramite il pulsante dedicato posizionato in basso accanto al tasto di invio.

• Rilasciare una **valutazione di gradimento a 5 stelle** con eventuale commento al termine della lavorazione.

Consente allo staff di supporto interno di:

• Monitorare le richieste nella **Dashboard con evidenziazione stile Outlook** (i ticket con nuove richieste o risposte da gestire sono evidenziati in grassetto con indicatore visivo e tornano a carattere normale non appena aperti o gestiti).

• Gestire prioritariamente stato, assegnazione tecnico e note di lavorazione grazie al **pannello operatore posizionato in evidenza** sopra il modulo di risposta.

• Acquisire e allegare **registrazioni audio/video di sessioni e chiamate** (con cattura microfono, condivisione schermo o upload file multimediali fino a 1 GB) ad uso strettamente interno.

• Disporre della **trascrizione vocale locale automatica (AI)** e dell'**analisi OCR dei fotogrammi video** nel pieno rispetto delle normative sulla privacy (GDPR compliant, zero dipendenze da cloud terzi).

• Tracciare SLA, consultare il rating a stelle e inviare notifiche email asincrone differenziate (salvataggio interno vs notifica cliente).

L'applicazione è strutturata con rendering lato server ad alte prestazioni (Django Templates + Tailwind CSS), integrata con file statici serviti da WhiteNoise e backend database MySQL in produzione (con supporto SQLite per lo sviluppo locale).

---

2. Dettaglio Componenti Tecnologici

2.1 Backend & Web Framework

Componente	Tecnologia / Versione	Note di Implementazione
Linguaggio	Python 3.12 / 3.14	Compatibile con ambienti CloudLinux/Passenger e server VPS dedicated
Framework Web	Django 6.1	Gestione modelli ORM, form validation, autenticazione e sicurezza CSRF
Server Applicativo (Prod)	Gunicorn 23.0+	Server WSGI di produzione multi-worker con monitoraggio PID
Server Statici	WhiteNoise 6.7.0	Compilazione e distribuzione ottimizzata dei file statici con cache e gzip
Database Driver	PyMySQL 1.1+	Driver nativo integrato con alias MySQLdb per Django
SQL Engine	sqlparse 0.6.0	Parsing ed esecuzione query ORM
Timezone	tzdata 2026.3	Fuso orario impostato su `Europe/Rome` (UTC+1/+2)

Speech-to-Text AI	faster-whisper 1.2+ / CTranslate2	Modello Whisper per trascrizione vocale locale senza dipendenze cloud
Media Processing	FFmpeg	Estrazione traccia audio mono 16kHz e campionamento keyframe video
OCR Engine	pytesseract 0.3+ / Tesseract OCR	Riconoscimento ottico caratteri (errori, IP, domini) sui frame video

---

2.2 Database & Persistenza

• **Ambiente Locale / Sviluppo**: SQLite (`db.sqlite3`), attivato automaticamente su Windows o con flag `USE_SQLITE=1`.

• **Ambiente di Produzione**: MySQL 8+ su host dedicato (`148.251.75.142:3306`), database `afterbit-test100` con codifica `utf8mb4` e strict transaction mode.

• **Modelli di Dominio Principali**:

• `Ticket`: entità centrale con campi `status`, `priority`, `categoria`, `servizio`, `referente`, `codice_riferimento`, `rating` (1-5), `rating_comment`, `rated_at`, `last_activity_at`.

• `TicketUpdate`: messaggi del thread di conversazione con supporto per autore (staff/cliente), nota testuale e `attachment` (`FileField` protetto con validazione estensione e limite 10MB).

• `TicketReadStatus`: tracciamento di lettura e apertura del ticket per singolo utente (`ticket`, `user`, `last_read_at`).

• `TicketRecording`: archivio registrazioni audio/video riservate al solo staff (`media_type`, `recording_file`, `duration_seconds`, `transcript`, `extracted_notes`, `status`, `error_message`).

• `UserProfile`: estensione del modello `User` con numero di telefono e dati aziendali.

• **Migrazioni**: 9 migrazioni attive e sincronizzate (inclusa la migrazione `0009_ticketrecording`).

2.3 Frontend & Interfaccia Utente

• **Architettura UI**: Server-Side Rendering (SSR) tramite Django Template Engine.

• **Design System**: Tailwind CSS con palette istituzionale Afterbit (Navy `#1C244B`, Orange `#EC3C11`, Slate `#324A6D`, Background `#F3F5F8`).

• **Funzionalità & Componenti UI**:

• **Dashboard Stile Outlook**: evidenziazione dinamica in grassetto (`font-bold text-[#1C244B]`) e indicatore circolare arancione ● per le richieste da gestire/non lette, con disattivazione automatica dopo l'apertura.

• **Layout Scheda Ticket Ottimizzato per Staff**: il pannello di gestione stato, assegnazione operatore e note rapide è posizionato prioritariamente prima della risposta.

• **Modulo Registrazione Media Live**: interfaccia HTML5 MediaRecorder per la cattura microfonica e la condivisione dello schermo in tempo reale con visualizzazione temporale e timer live.

• **Promemoria GDPR in Evidenza**: banner ad alto contrasto con testo obbligatorio per l'acquisizione del consenso informato dell'utente.

• **Doppio Pulsante Azione Cliente**: posizionamento affiancato in basso dei pulsanti `[ ■ Chiudi Ticket ]` e `[ ■ Invia Messaggio ]`.

• **Thread di Conversazione**: timeline stile chat con badge distintivi per cliente e supporto tecnico.

• **Widget Feedback a 5 Stelle**: componente interattivo a stelle con hover visivo, preselezione immediata e campo commenti.

• **Combobox Categoria Specifica**: ricerca istantanea alla digitazione della prima lettera e navigazione tastiera (↓/↑/Enter).

• **Ordinamento Dinamico**: sorting interattivo delle colonne della Dashboard tramite parametri sanitizzati.

• **Modal di Chiusura Rapida**: finestra modale per la conferma di chiusura autonoma con motivazione facoltativa.

2.4 Trascrizione Vocale Locale, Elaborazione Media & OCR

• **Architettura AI Locale (Privacy by Design)**:

• Motore basato su `faster-whisper` (implementazione reimplementata di OpenAI Whisper tramite CTranslate2).

• Quantizzazione `int8` su CPU standard o `float16` su GPU NVIDIA CUDA per massime performance.

• Cache del modello in memoria (`_MODEL_CACHE`) per evitare il caricamento da disco a ogni singola richiesta.

• Filtraggio attivo dei silenzi prolungati tramite algoritmo VAD (*Voice Activity Detection*) e generazione di trascrizioni temporizzate (`[MM:SS - MM:SS]`).

• **Pipeline Elaborazione Video & FFmpeg**:

• Riconoscimento automatico del formato video (`.mp4`, `.webm`, `.mov`, `.mkv`).

• Estrazione stream audio dedicato con ricampionamento PCM a 16.000 Hz mono per la massima accuratezza di trascrizione.

• Estrazione automatica dei fotogrammi significativi (fps=1/45) per la documentazione visiva dell'intervento.

• **Motore di Analisi OCR (Tesseract / Pytesseract)**:

• Scansione OCR multilingua (`ita+eng`) su ciascun fotogramma estratto.

• Riconoscimento intelligente tramite pattern matching regex e blacklist/whitelist:

• Messaggi di errore critici (*fatal error*, *exception*, *error 500/404*, *timeout*, *access denied*, *stack trace*).

• Indirizzi IP e porte rilevati a video.

• URL e domini visualizzati durante la sessione.

• **Elaborazione Asincrona & Management Command**:

• Dispatching asincrono non bloccante via thread daemon (`queue_recording_transcription`).

• Management command Django dedicato: `python manage.py transcribe_recordings` con opzioni `--id` e `--retry-failed` per esecuzioni batch e recuperi automatici.

2.5 Autenticazione, Permessi & Sicurezza

• **Sistema Utenti**: `django.contrib.auth` con doppio livello di autorizzazione:

• *Cliente*: accesso ai propri ticket, invio repliche, caricamento allegati, chiusura autonoma e rilascio valutazione.

• *Staff/Operatore* (`is_staff = True`): accesso alla gestione globale, cambio stati, assegnazione, note tecniche, registrazione audio/video e consultazione trascrizioni/OCR.

• **Riservatezza Registrazioni Interne**: i modelli e gli endpoint legati a `TicketRecording` sono confinati categoricamente agli utenti aventi `is_staff = True`.

• **Registrazione Clienti**: validazione email univoca, associazione profilo telefonico e verifica di sicurezza anti-bot tramite CAPTCHA matematico dinamico in sessione.

• **Protezione Dati & Privacy**: la pagina pubblica di verifica ticket (`/ticket/cerca/`) applica la doppia verifica obbligatoria (ID/Codice ticket + corrispondenza email/referente/telefono).

• **Protezione CSRF**: middleware Django con validazione token e supporto cookie su proxy inverso.

2.6 Gestione Notifiche Email

• **Server SMTP**: `smtp.afterbit.it` (Porta 587 con cifratura TLS / STARTTLS o 465 SSL).

• **Mittente Ufficiale**: `Afterbit Assistenza .`

• **Indirizzo di Notifica Interno**: `info@afterbit.com`.

• **Eventi Notificati**:

• Ricevuta di apertura ticket per cliente e staff.

• Nuova risposta del cliente nel ticket (notifica immediata allo staff).

• Aggiornamento di stato o risposta dello staff al cliente.

• Chiusura intervento con invito alla valutazione del servizio a 5 stelle.

• **Esecuzione Asincrona**: invio email delegato a thread daemon in background con timeout di 5 secondi per garantire tempi di risposta HTTP immediati (< 100ms).

2.7 Servizi Esterni & FAQ

• **Geolocalizzazione IP**: interroga l'API `ip-api.com` all'apertura del ticket con timeout di 3 secondi e fallback sicuro.

• **Base di Conoscenza FAQ**: 10 categorie di supporto complete con sezione dedicata (FAQ 7) per la configurazione dei server email Afterbit:

• **IMAP**: `mail.afterbit.it` (Porta SSL 993 / No SSL 143)

• **POP3**: `mail.afterbit.it` (Porta SSL 995 / No SSL 110)

• **SMTP**: `smtp.afterbit.it` (Porta SSL 465 / No SSL 25)

• **PEC**: parametri certificati IMAP (`imaps.pec.aruba.it`: 993) e SMTP (`smtps.pec.aruba.it`: 465).

2.8 Architettura di Deployment & Hosting

• **Server di Produzione**: https://test100.afterbit.it

• **Punto di Ingresso Web**:

• Reverse proxy ottimizzato (`index.php`) con supporto completo per richieste HTTP `multipart/form-data` (upload file via `CURLFile`) e inoltro a Gunicorn (`127.0.0.1:8000`).

• Supporto per **Phusion Passenger** tramite [`passenger_wsgi.py`](file:///c:/Users/Utente/.gemini/antigravity/scratch/afterbit_ticket/passenger_wsgi.py).

• **Storage File & Media**:

• `media/ticket_attachments/`: allegati alle discussioni (max 10 MB per file con verifica estensioni consentite).

• `media/ticket_staff_recordings/%Y/%m/`: registrazioni audio/video riservate al solo staff (fino a 1 GB con verifica formati multimediali).

• **Testing**: Suite di 60 test automatizzati (`tickets/tests.py`) a copertura di modelli, viste, form, lettura ticket, registrazioni staff, elaborazione asincrona e permessi.

---

3. Struttura del Repository

```
afterbit_ticket/
■■■ afterbit_project/          # Configurazione centrale Django
■   ■■■ settings.py           # Impostazioni database, email, security e statici
■   ■■■ urls.py               # Routing primario degli endpoint
■   ■■■ wsgi.py               # WSGI entry point standard
■   ■■■ asgi.py               # ASGI entry point
■■■ tickets/                  # Applicazione Helpdesk
■   ■■■ models.py             # Modelli Ticket, UserProfile, TicketUpdate, TicketReadStatus, TicketRecording
■   ■■■ views.py              # Logica dashboard, thread risposte, feedback, upload registrazioni staff
■   ■■■ forms.py              # Form creazione, risposte con allegati, feedback 5 stelle, upload recording
■   ■■■ emails.py             # Dispatcher email asincrono (apertura, risposta, chiusura)
■   ■■■ service_transcription.py # Motore di trascrizione locale faster-whisper, FFmpeg e analisi OCR
■   ■■■ utils.py              # Geolocalizzazione IP, CAPTCHA, permessi
■   ■■■ tests.py              # Suite di 60 test automatizzati
■   ■■■ admin.py              # Configurazione del pannello Django Admin
■   ■■■ templates/            # Template HTML (Dashboard, Dettaglio Ticket, FAQ, Auth)
■   ■■■ management/           # Comandi personalizzati Django
■   ■   ■■■ commands/
■   ■       ■■■ transcribe_recordings.py # Comando CLI per trascrizione batch o rigenerazione
■   ■■■ migrations/           # Storico schema database (9 migrazioni sincronizzate)
■■■ static/                   # File statici compilati per la produzione
■■■ media/                    # Cartella protetta di upload allegati e registrazioni ticket
■■■ files/                    # Documenti PDF (Stack Tecnologico e Manuale Operatore)
■■■ passenger_wsgi.py         # Entry point per cPanel / CloudLinux Passenger
■■■ requirements.txt          # Dipendenze Python di produzione
■■■ Manuale_Operatore_Ticket.md # Manuale operativo per lo staff
■■■ Istruzioni_Pubblicazione_cPanel.md # Guida al deployment hosting
■■■ Stack_Tecnologico.md      # Questo documento
```